UNIVERSITATEA TEHNICĂ A MOLDOVEI
FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ
DEPARTAMENTUL INGINERIA SOFTWARE ȘI AUTOMATICĂ

Lucrare de laborator Nr. 6

Tehnici de Programare

Tema: Informații invizibile: forensics și steganografie (Decoderul Universal)

Realizat: **SI-264 Pavalachi Veronica**

Verificat: **Asistent Universitar Reitman Patricia**

Chișinău, 2026

A1-Incalzirea -Diferenta dintre text (str) si octeti (bytes)

```python
# A1: Încălzirea - Diferența dintre Text (str) și Octeți (bytes)
s = "salut"
b= s.encode()  # Convertim șirul în octeți (bytes)

print(b)                    # Afișează b"salut"
print(b[0])                 # Afișează valoarea numerică a primului octet (115 pentru 's')
print(b.hex())              # Afișează reprezentarea hexazecimală a octeților
print(bytes.fromhex(b.hex())) # Reface octeții înapoi din hexazecimal
```

Fig.1.1

Explicatie:In aceasta linie transforma,octetii in format hexazecimali,apoi ii transformam inapoi in octeti si afiseaza rezultatul “b’salut”.

A2-Recunoasterea stratului dupa forma (Ghicirea codarii)

```python
# A2: Recunoașterea stratului după formă (Ghicirea codării)
def ghici_strat(s :{__len__} ):  4 usages
    # Verificăm întâi formatul URL (%xx) - cel mai specific
    if re.search( pattern: r"%[0-9a-fA-F]{2}", s):
        return "url"

    # Verificăm dacă este Hex pur (caractere 0-9, a-f și lungime pară)
    if re.fullmatch( pattern: r"[0-9a-fA-F]+", s) and len(s) % 2 == 0:
        return "hex"

    # Verificăm dacă este Base64 (alfabet standard, lungime multiplu de 4 cu padding '=')
    if re.fullmatch( pattern: r"[A-Za-z0-9+/]+={0,2}", s) and len(s) % 4 == 0:
        return "base64"

    return "necunoscut"

# Teste pentru funcția de ghicire
print(ghici_strat("%2Fetc"))          # -> url
print(ghici_strat("48656c6c6f"))      # -> hex
print(ghici_strat("U2FsdXQ="))        # -> base64

import base64
from urllib.parse import unquote
```

Fig. 1.2

Explicatie: Am scris o funcție cu expresii regulate (re) care analizează textul și ghicește singură dacă este codat în **URL** (are %),**Hexadecimal** sau **Base64**.

A3-Desfacerea stratului recunoscut

```
# A3: Desfacerea stratului recunoscut
# aplicăm funcția inversă corespunzătoare pentru a decoda șirul în octeți.
def desfa(s :{__len__}):  6 usages
    strat = ghici_strat(s)

    if strat == "base64":
        rezultat = base64.b64decode(s)
    elif strat == "hex":
        rezultat = bytes.fromhex(s)
    elif strat == "url":
        rezultat = unquote(s).encode()
    else:
        rezultat = s.encode()

    return strat, rezultat

print(desfa("U2FsdXQ="))
print(desfa("48656c6c6f"))
print(desfa("%2Fetc"))
```

Fig .1.3

Explicatie: În funcție de ce a recunoscut pasul anterior, programul a aplicat decodarea corectă pentru a aduce datele la forma lor inițială.

A4-Straturi multiple intr-o bucla (Citirea din ‘mesaj.txt’)

```
# A4: Straturi multiple într-o buclă (Citirea din 'mesaj.txt')
# (de ex. Base64 peste Hex). Facem o buclă de până la 8 pași care decodifică iterativ până la textul clar.
with open("mesaj.txt") as fisier:
    mesaj = fisier.read().strip()

for i in range(8):
    strat, rezultat = desfa(mesaj)

    if strat == "necunoscut":
        break  # Oprim bucla dacă nu mai recunoaște nicio codare

    mesaj = rezultat.decode()
    print(strat, mesaj)

print("Mesaj final:", mesaj)
```

Fig .1.4

Explicatie: Am pus decodarea într-o buclă repetitivă (de cel mult 8 ori) pentru a desface un mesaj ascuns în mai multe straturi suprapuse (Base64 și Hex) până la textul clar.

A5-Spargerea unui XOR cu cheie de un octet (‘xor.bin’)

```
# A5: Spargerea unui XOR cu cheie de un octet ('xor.bin')
# Testăm toate cheile prin operatorul XOR (^), căutând markerul standard 'FLAG{' în textul rezultat.
with open("xor.bin", "rb") as fisier:
    date = fisier.read()

for cheie in range(256):
    clar = bytes(b ^ cheie for b in date)

    if b"FLAG{" in clar:
        print("Cheie gasita:", cheie)
        print("Mesaj decriptat:", clar.decode())
        break
```

Fig1.5

Explicatie: Am spart un fișier criptat cu o cheie pe un singur octet testând toate cele **256 de variante posibile** (forță brută), oprindu-ne automat când am găsit textul care începea cu FLAG{.

A6-Spargere unui hash prin dictionar (‘hashuri.txt’ + ‘wordlist.txt’)

```
# A6: Spargerea unui hash prin dicționar ('hashuri.txt' + 'wordlist.txt')
# sau dicționar: hashuim fiecare cuvânt din wordlist și comparăm rezultatul cu hashul țintă.
with open("hashuri.txt") as fisier:
    tinta = fisier.readline().strip()

print("Lungimea hashului:", len(tinta)) # 32 caractere indică algoritmul MD5

with open("wordlist.txt") as fisier:
    for parola in fisier:
        parola = parola.strip()
        calculat = hashlib.md5(parola.encode()).hexdigest()

        if calculat == tinta:
            print("Parola gasita:", parola)
            break
```

Fig .1.6

Explicatie: Am luat un cod de tip MD5 și l-am căurs într-un dicționar (wordlist.txt), calculând hash-ul fiecărui cuvânt până am găsit potrivirea exactă (deoarece hash-urile nu se pot "decripta" direct).

A7-Argumente in linie de comanda (CLI) salvare in JSON si Test Unitar

```python
# A7: Argumente în linie de comandă (CLI), salvare în JSON și Test Unitar
# primind un șir sau un fișier, salvăm rezultatul într-o fișă JSON stru ată și testăm validitatea cu assert.
p = argparse.ArgumentParser()
p.add_argument("intrare")
p.add_argument( *name_or_flags: "--fisier", action="store_true")
a = p.parse_args()

if a.fisier:
    with open(a.intrare, "rb") as fisier:
        mesaj = fisier.read().decode().strip()
else:
    mesaj = a.intrare

strat, rezultat = desfa(mesaj)

# Salvăm rezultatul într-o fișă JSON oficială
fisa = {
    "strat": strat,
    "rezultat": rezultat.decode()
}

with open("fisa.json", "w", encoding="utf-8") as fisier:
    json.dump(fisa, fisier, indent=4, ensure_ascii=False)

print("Rezultatul a fost salvat in fisa.json")

# Test unitar pentru verificarea corectitudinii funcției de Base64
def test_base64():  1 usage
    strat, rezultat = desfa("U2FsdXQ=")
    assert strat == "base64"
    assert rezultat == b"Salut"

test_base64()
print("Testul Base64 a trecut!")
```

Fig .1.7

Explicatie: Am transformat programul într-o unealtă care se poate rula din linia de comandă folosind argparse, am salvat automat rezultatul într-un fișier numit fisa.json (care conține tipul stratului detectat și textul rezultat în clar {"strat": "base64", "rezultat": "Salut"}) și am scris un test unitar cu assert pentru a ne asigura că totul funcționează corect.

REZULTATUL

```
/Users/pavalacheveronika/PycharmProjects/WelcomeScreen/probe/.venv/bin/python /Users/pavalacheveronika/PycharmProjects/WelcomeScreen/probe/decode.py U2FsdXQ=
b'salut'
115
73616c7574
b'salut'
url
hex
base64
('base64', b'Salut')
('hex', b'Hello')
('url', b'/etc')
base64 NDY0YzQxNDc3YjczNzQ3MjYxNzQ3NTcyNjk1ZjY0NjU1ZjY1NmU2MzZmNjQ2OTZlNjc3ZA==
base64 464c41477b737472617475726 95f64655f656e636f64696e677d
hex FLAG{straturi_de_encoding}
Mesaj final: FLAG{straturi_de_encoding}
Cheie gasita: 128
Mesaj decriptat: Un singur octet tine tot secretul... gaseste-l! FLAG{xor_un_octet}
Lungimea hashului: 32
Parola gasita: password
Rezultatul a fost salvat in fisa.json
Testul Base64 a trecut!

Process finished with exit code 0
```



B1: Magic Bytes (Detectarea tipului real al fișierului)

```python
import re
from PIL import Image
from PIL.ExifTags import TAGS
import zipfile
import io


# B1: Magic bytes (Detectarea tipului real al fișierului)
# Verificăm primii octeți din antetul fișierului deschis în binar pentru a afla tipul lui real, indiferent de extensie.
def tip_real(cale):  1 usage
    with open(cale, "rb") as f:
        cap = f.read(8)

    if cap.startswith(b"%PDF"):
        return "pdf"
    elif cap.startswith(b"\x89PNG"):
        return "png"
    elif cap.startswith(b"\xff\xd8"):
        return "jpeg"
    elif cap.startswith(b"PK\x03\x04"):
        return "zip/docx"
    return cap.hex()


print("Tip real foto.jpg:", tip_real("foto.jpg"))
```

Fig .2.1

Explicatie: Citim primii octeți din fișier pentru a-i afla formatul real, indiferent de extensia pe care o are pe disc.

B2-Strings (Extragerea textului citibil dintr-un binar)

```
# B2: Strings (Extragerea textului citibil dintr-un binar)
# Folosim o expresie regulată pe octeți pentru a extrage toate secvențele de text vizibile ascunse în interiorul unui fișier binar.
with open("ascuns.png", "rb") as f:
    date_binare = f.read()

print("--- Text extras din ascuns.png (Strings) ---")
for m in re.findall( pattern: rb"[ -~]{4,}", date_binare):
    print(m.decode(errors="ignore"))
```

Fig. 2.2

Explicatie: Scanăm întregul fișier binar folosind expresii regulate pentru a găsi și extrage toate secvențele de text vizibile și caracterele imprimabile care ar putea conține mesaje sau indicii ascunse.

B3-Metatade EXIF si GPS din poza

```
# B3: Metadate EXIF și GPS din poză
# ----------------------------------------------------------
# Deschidem imaginea cu biblioteca PIL și extragem dicționarul EXIF
img = Image.open("foto.jpg")
exif_data = img._getexif()

if exif_data:
    print("--- Metadate EXIF & GPS ---")
    for tag_id, val in exif_data.items():
        tag = TAGS.get(tag_id, tag_id)
        if tag == "GPSInfo":
            print(f"Coordonate GPS (Tag 34853): {val}")
        else:
            print(f"{tag}: {val}")
```

Fig.2.3

Explicatie: Deschidem imaginea cu ajutorul bibliotecii PIL pentru a citi informațiile ascunse în structura ei, cum ar fi detaliile despre dispozitivul folosit la fotografiere, data, ora și coordonatele geografice (GPS) exacte ale locului.

B4-File carving (Extragerea arhivei ZIP ascune)

```
# B4: File carving (Extragerea arhivei ZIP ascunse)
# ------------------------------------------------------
# Căutăm semnătura unei arhive ZIP (PK\x03\x04) care a fost lipită ascuns la coa
with open("ascuns.png", "rb") as f:
    continut = f.read()

i = continut.find(b"PK\x03\x04")
if i != -1:
    with open("gasit.zip", "wb") as f_out:
        f_out.write(continut[i:])
    print(f"Arhivă ZIP extrasă cu succes de la octetul {i} în 'gasit.zip'!")
```

Fig.2.4

Explicatie: Căutăm în interiorul unei imagini sau al unui alt tip de fișier semnătura binară specifică a unei arhive ZIP, iar când o descoperim, o decupăm de la acel octet și o salvăm separat pe disc ca fișier nou.

B5-Ascunderea unui mesaj prin LSB (Least Significant Bit)

```
# B5: Ascunderea unui mesaj prin LSB (Least Significant Bit)
# Transformăm mesajul în biți și îi ascundem în ultimul bit al c
img_stego = Image.open("foto.jpg").convert("RGB")
px = list(img_stego.getdata())

mesaj = b"FLAG{ascuns_in_pixeli}\x00"  # Mesajul de ascuns
biti = "".join(f"{x:08b}" for x in mesaj)

px_nou = []
bit_idx = 0
for p in px:
    r, g, b = p
    if bit_idx < len(biti):
        r = (r & ~1) | int(biti[bit_idx])
        bit_idx += 1
    if bit_idx < len(biti):
        g = (g & ~1) | int(biti[bit_idx])
        bit_idx += 1
    if bit_idx < len(biti):
        b = (b & ~1) | int(biti[bit_idx])
        bit_idx += 1
    px_nou.append((r, g, b))
else:
    px_nou.extend(px[len(px_nou):])

img_stego.putdata(px_nou)
img_stego.save("stego.png")
print("Mesaj ascuns și salvat în 'stego.png'!")
```

Fig.2.5

Explicatie: Transformăm un mesaj text în biți și îi ascundem discret în ultimul bit (cel mai puțin semnificativ) al culorilor fiecărui pixel din imagine, modificând atât de puțin culorile încât ochiul uman nu poate observa nicio diferență.

B6-Dezvaluirea mesajului LSB din stego.png

```
# B6: Dezvăluirea mesajului LSB din stego.png
# Citim ultimul bit din fiecare pixel al imaginii modificate, îi grupăm câte 8 înapoi
img_citire = Image.open("stego.png").convert("RGB")
px_cititi = list(img_citire.getdata())

biti_extras = ""
for (r, g, b) in px_cititi:
    biti_extras += str(r & 1)
    biti_extras += str(g & 1)
    biti_extras += str(b & 1)

octeti = bytearray()
for b_index in range(0, len(biti_extras), 8):
    octet_str = biti_extras[b_index:b_index + 8]
    if len(octet_str) < 8:
        break
    valoare = int(octet_str, 2)
    if valoare == 0:  # Sfârșitul mesajului
        break
    octeti.append(valoare)

print("Mesaj dezvăluit din LSB:", octeti.decode(errors="ignore"))
```

Fig.2.6

Explicatie: Citind ultimul bit din fiecare pixel al imaginii modificate, extragem șirul de biți ascunși, îi grupăm câte opt pentru a reconstitui octeții și transformăm totul înapoi în textul secret original.

REZULTATUL

```
/Users/pavalacheveronika/PycharmProjects/WelcomeScreen/probe/.venv/bin/python /Users/pavalacheveronika/PycharmProjects/WelcomeScreen/probe/forensics.py
Tip real foto.jpg: jpeg
--- Text extras din ascuns.png (Strings) ---
IHDR
IDATx
IEND
secret.txtFLAG{ascuns_prin_carving}
secret.txtPK
--- Metadate EXIF & GPS ---
Model: iPhone 13
DateTime: 2026:09:14 10:30:00
Coordonate GPS (Tag 34853): {1: 'N', 2: (47.0, 1.0, 0.0), 3: 'E', 4: (28.0, 51.0, 27.0)}
Make: Apple
Arhivă ZIP extrasă cu succes de la octetul 782 în 'gasit.zip'!
Mesaj ascuns și salvat în 'stego.png'!
Mesaj dezvăluit din LSB: FLAG{ascuns_in_pixeli}

Process finished with exit code 0
```

Concluzia finala

În cadrul acestui laborator am aplicat concepte practice de informatică forensică și steganografie folosind Python. Am învățat cum să detectăm tipul real al fișierelor, cum să extragem metadate și text ascuns, cum să recuperăm arhive pierdute prin *file carving* și cum să ascundem sau să citim mesaje secrete în pixeli prin steganografie LSB. Astfel, am înțeles mai bine cum pot fi investigate, descoperite și protejate informațiile în medii digitale.